

딥러닝 코드

Ch.02 인공신경망 (ANN)

▶ Fashion MNIST 데이터 로드 & 분할

```
from tensorflow import keras
from sklearn.model_selection import train_test_split

(train_input, train_target), (test_input, test_target) = \
    keras.datasets.fashion_mnist.load_data()

train_scaled = train_input / 255.0
train_scaled = train_scaled.reshape(-1, 28*28)

train_scaled, val_scaled, train_target, val_target = \
    train_test_split(train_scaled, train_target, test_size=0.2)
```

▶ ANN 모델 생성 & 훈련

```
dense = keras.layers.Dense(10, activation='softmax',
                             input_shape=(784,))
model = keras.Sequential([dense])

model.compile(loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(train_scaled, train_target, epochs=5)
model.evaluate(val_scaled, val_target)
```

Ch.03 심층신경망 (DNN)

▶ DNN 모델 구성 (여러 방법)

```
# 방법1: Sequential에 리스트 전달
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)),
    keras.layers.Dense(100, activation='relu'),
    keras.layers.Dense(10, activation='softmax')
], name='fashion_model')

# 방법2: add()로 층 추가
model = keras.Sequential()
model.add(keras.layers.Flatten(input_shape=(28, 28)))
model.add(keras.layers.Dense(100, activation='relu'))
model.add(keras.layers.Dense(10, activation='softmax'))

model.summary()
```

▶ 옵티마이저 지정 & 비교

```
# SGD + 네스테로프 모멘텀
model.compile(
    optimizer=keras.optimizers.SGD(
        learning_rate=0.01, momentum=0.9, nesterov=True),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy'])

# Adam (가장 범용적으로 좋은 성능)
model.compile(optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy'])
```

Ch.04 신경망 훈련 테크닉

▶ 검증 데이터 포함 훈련 & 손실곡선

```
history = model.fit(train_scaled, train_target,
                    epochs=20,
                    validation_data=(val_scaled, val_target))

import matplotlib.pyplot as plt
plt.plot(history.history['loss'], label='train')
plt.plot(history.history['val_loss'], label='val')
plt.xlabel('epoch'); plt.ylabel('loss')
plt.legend(); plt.show()
```

▶ 드롭아웃 적용

```
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)),
    keras.layers.Dense(100, activation='relu'),
    keras.layers.Dropout(0.3), # 30% 뉴런 드롭아웃
    keras.layers.Dense(10, activation='softmax')
])
```

▶ 콜백 & 조기종료

```
checkpoint_cb = keras.callbacks.ModelCheckpoint(
    'best-model.keras', save_best_only=True)

early_stopping_cb = keras.callbacks.EarlyStopping(
    patience=2, restore_best_weights=True)

history = model.fit(train_scaled, train_target,
                    epochs=20,
                    validation_data=(val_scaled, val_target),
                    callbacks=[checkpoint_cb, early_stopping_cb])

print(early_stopping_cb.stopped_epoch)
```

▶ 모델 저장 & 복원

```
# 가중치만 저장/복원
model.save_weights('model_weights.h5')
model.load_weights('model_weights.h5')

# 모델 전체 저장/복원
model.save('model_whole.h5')
loaded = keras.models.load_model('model_whole.h5')

# 예측
import numpy as np
preds = model.predict(val_scaled)
classes = np.argmax(preds, axis=-1)
```

Ch.05 합성곱 (Convolution)

▶ 합성곱 & 풀링 층 정의

```
keras.layers.Conv2D(
    32,                      # 필터 개수
    kernel_size=(3, 3),      # 커널 크기
    activation='relu',
    padding='same',          # 세임 패딩
    strides=1,
    input_shape=(28, 28, 1)) # 첫 층만 지정

keras.layers.MaxPooling2D(2) # (2,2) 최대 풀링
keras.layers.AveragePooling2D(2) # 평균 풀링
```

Ch.06 CNN 이미지 분류

▶ CNN 전체 모델 (Fashion MNIST)

```
train_scaled = train_scaled.reshape(-1, 28, 28, 1)

model = keras.Sequential([
    # 합성곱 블록 1
    keras.layers.Conv2D(32, kernel_size=3,
        activation='relu', padding='same',
        input_shape=(28, 28, 1)),
    keras.layers.MaxPooling2D(2),      # → (14,14,32)
    # 합성곱 블록 2
    keras.layers.Conv2D(64, kernel_size=3,
        activation='relu', padding='same'),
    keras.layers.MaxPooling2D(2),      # → (7,7,64)
    # 분류기
    keras.layers.Flatten(),            # → 3136
    keras.layers.Dense(100, activation='relu'),
    keras.layers.Dropout(0.4),
    keras.layers.Dense(10, activation='softmax')
])
```

▶ CNN 컴파일 & 훈련

```
model.compile(optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy'])

checkpoint_cb = keras.callbacks.ModelCheckpoint(
    'best-cnn-model.keras', save_best_only=True)
early_stopping_cb = keras.callbacks.EarlyStopping(
    patience=2, restore_best_weights=True)

history = model.fit(train_scaled, train_target,
    epochs=20,
    validation_data=(val_scaled, val_target),
    callbacks=[checkpoint_cb, early_stopping_cb])

keras.utils.plot_model(model, show_shapes=True)
```

Ch.09 IMDB 리뷰 분류 (RNN)

▶ IMDB 데이터 준비

```
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences

(train_input, train_target), (test_input, test_target) = \
    imdb.load_data(num_words=300)

train_seq = pad_sequences(train_input, maxlen=100)
```

▶ 원-핫 인코딩 RNN 모델

```
from tensorflow.keras.utils import to_categorical

train_oh = to_categorical(train_seq) # (25000, 100, 300)

model = keras.Sequential([
    keras.layers.SimpleRNN(8, input_shape=(100, 300)),
    keras.layers.Dense(1, activation='sigmoid')
])
# 파라미터:  $300 \times 8 + 8 \times 8 + 8 = 2472$ 
```

▶ 단어 임베딩 RNN 모델

```
model = keras.Sequential([
    keras.layers.Embedding(300, 16, input_length=100),
    keras.layers.SimpleRNN(8),
    keras.layers.Dense(1, activation='sigmoid')
])
# Embedding:  $300 \times 16 = 4800$ , RNN:  $16 \times 8 + 8 \times 8 + 8 = 200$ 
# Dense:  $9 \rightarrow$  합계: 5009

rmsprop = keras.optimizers.RMSprop(learning_rate=1e-4)
model.compile(optimizer=rmsprop,
              loss='binary_crossentropy', metrics=['accuracy'])

history = model.fit(train_seq, train_target,
                    epochs=100, batch_size=64,
                    validation_data=(val_seq, val_target),
                    callbacks=[checkpoint_cb, early_stopping_cb])
```

Ch.10 LSTM & GRU

▶ LSTM 모델

```
model = keras.Sequential([
    keras.layers.Embedding(500, 16, input_length=100),
    keras.layers.LSTM(8),
    keras.layers.Dense(1, activation='sigmoid')
])
# Embedding: 500×16=8000
# LSTM: (16×8 + 8×8 + 8)×4 = 800
# Dense: 9 → 합계: 8809
```

▶ 2개의 순환층 연결 (+ 드롭아웃)

```
model = keras.Sequential([
    keras.layers.Embedding(500, 16, input_length=100),
    keras.layers.LSTM(8, dropout=0.3,
        return_sequences=True), # 모든 타임스텝 출력!
    keras.layers.LSTM(8, dropout=0.3), # 마지막만 출력
    keras.layers.Dense(1, activation='sigmoid')
])
# 마지막 순환층 제외 모든 순환층에
# return_sequences=True 필수
```

▶ GRU 모델

```
model = keras.Sequential([
    keras.layers.Embedding(500, 16, input_length=100),
    keras.layers.GRU(8),
    keras.layers.Dense(1, activation='sigmoid')
])
# GRU: 작은 셀 3개 → LSTM보다 파라미터 적음
```